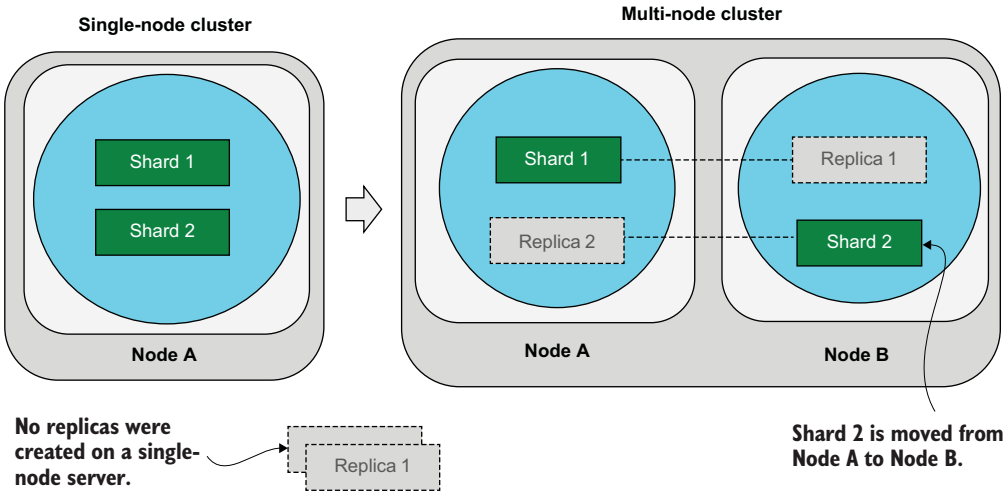


Figure 14.5 A newly joined node gets new shards moved from the single-node cluster.

Should node A crash, replica 1 on node B is instantly promoted to shard 1 and thus goes back to the single-node cluster until the second node is back online. We can keep adding new nodes to the cluster when necessary by bringing up the Elasticsearch server on the new node.

14.1.3 Increasing read throughput

Increasing the number of replicas has an added performance benefit. The replicas increase read throughput: reads (queries) can be served by the replicas while the shards do the indexing operations. If an application is read-intensive (that is, an application gets higher volumes of search queries than volumes of data that it indexes, typically an e-commerce application), increasing the number of replicas alleviates the load on the application.

As each replica is an exact copy of the shard, we can split the application's shards into two categories: *primary shards* deal with data, and *replica shards* deal with the read side of data. When a query comes from the client, the coordinator node diverts that request onto the read nodes for an answer. Thus, adding more replicas to the index helps improve read throughputs. Note the memory implications, however: because each replica is an exact copy of the shard, we need to size the cluster accordingly (see section 14.3).

To increase the replicas (and thus read throughput) to counter read-query performance bottlenecks, one strategy is to update the `number_of_replicas` setting on the live index. This is a dynamic setting that we can tweak even when the index is live and in production. For example, suppose we add 10 nodes to our 5-node cluster, with five shards to counteract read-query performance problems that are bringing the server to